

Como corrigir as vulnerabilidades do Synergia

Integrantes do grupo: Guilherme Harmatiuk Arantes **Data:** 18/09/2026 **Base:** Relatório de Auditoria de Segurança (relatorio-auditoria-seguranca.pdf)

Introdução

A auditoria encontrou 24 problemas de segurança no Synergia. A maioria tem a mesma origem: **o servidor não sabe quem está fazendo cada pedido**. Depois do login, o navegador informa sozinho qual é o usuário e a instituição, e o servidor acredita. Por isso, qualquer pessoa consegue ler, alterar e apagar dados de qualquer instituição.

Este documento explica o que fazer para corrigir cada grupo de problemas e em que ordem. Os códigos entre parênteses (C-01, A-03 etc.) são os mesmos do relatório.

1. Criar uma autenticação de verdade no servidor

Problema (C-01): nenhuma rota da API exige identificação. Qualquer pessoa na internet pode chamar a API diretamente, sem passar pelo site.

Como resolver:

- Adicionar o Spring Security ao backend.
- No login, o servidor passa a entregar um token assinado, que o navegador envia em todos os pedidos seguintes.
- O servidor recusa qualquer pedido sem token válido. Só ficam abertas as telas que realmente precisam ser públicas, como o login e o cadastro de instituição.
- A chave usada para assinar o token fica numa variável de ambiente, nunca no código.

Essa é a correção mais importante: quase todas as outras dependem dela.

2. Corrigir o login

Problema (C-02): existe um modo de login em que o navegador avisa "esse usuário entrou há pouco" e o servidor deixa entrar **sem pedir a senha**. Com isso, dá para entrar na conta de qualquer pessoa que usou o sistema nas últimas 12 horas, inclusive o administrador.

Como resolver:

- Remover esse modo. O login deve sempre conferir a senha.
- Para o recurso "continuar logado", usar um token de renovação criado pelo servidor, com prazo de validade e guardado de forma segura no navegador.
- Alterar o teste automático que hoje garante o comportamento errado, para que ele passe a exigir que o login sem senha seja recusado.

3. Isolar os dados de cada instituição (banco sem tranca)

Problema (A-01, A-02, M-01, M-03, B-01): as listas de contas, eventos, projetos e tags já filtram por instituição, mas quem escolhe a instituição é o próprio navegador. Basta trocar o número para ver os dados de outra. A lista de instituições também mostra as privadas.

Como resolver:

- Depois da correção 1, o servidor passa a pegar a instituição do usuário a partir do token, e não mais do pedido.
- Aplicar o mesmo filtro de instituição nas consultas que hoje filtram só por evento, projeto ou conta.
- A lista pública de instituições deve esconder as privadas e mostrar só o necessário para o login.
- A consulta "esse login ou e-mail existe?" só deve funcionar para usuários logados da mesma instituição.

4. Verificar permissões no servidor, não só na tela

Problema (A-03, A-04, I-01): o único controle de acesso fica no navegador. O menu de Administração aparece para todos, e o servidor não confere se o usuário tem permissão. Além disso, o autor e a instituição de um novo evento ou projeto são informados pelo navegador, então dá para criar registros em nome de outras pessoas. As permissões do sistema (como "criar usuário") existem no código, mas nunca são usadas.

Como resolver:

- Usar as permissões que já existem para bloquear, no servidor, as ações administrativas de quem não as tem.
- O autor e a instituição de qualquer registro novo passam a vir do token do usuário, e não do formulário.
- Só depois disso, esconder o menu de Administração para quem não tem permissão. Esconder na tela é conforto visual; a proteção real é a do servidor.

- As funções de atribuir permissões e ligar registros, que ainda não foram terminadas, devem ser concluídas já com essa verificação.

5. Conferir o dono antes de ler, alterar ou apagar (IDOR)

Problema (C-03, A-05, A-06, A-07, M-02): as rotas que recebem um número de registro (conta, evento, projeto, tag, instituição) fazem a ação sem conferir se aquele registro pertence a quem pediu. O caso mais grave é a edição de conta, que permite **trocar a senha do administrador de qualquer instituição**. A exclusão de tags ainda funciona por um simples acesso a um link.

Como resolver:

- Criar uma verificação única, usada em todas essas rotas: buscar o registro pelo número e pela instituição do usuário. Se não for dele, responder "não encontrado".
- Edição de eventos e projetos: só para membros, organizadores ou quem tem a permissão de editar como não membro.
- Edição de contas: só a própria conta ou um administrador da mesma instituição. Para trocar a própria senha, pedir a senha atual.
- Edição de instituição: só para o administrador dela.
- Trocar a exclusão de tag para o método de exclusão correto, que não é disparado apenas por abrir um link.

6. Tirar as senhas e chaves do código

Problema (A-08, M-04, B-02, B-03): o repositório é público e contém um token com permissão de escrita no Nx Cloud (serviço de cache de build), a senha do banco de desenvolvimento repetida em vários arquivos e na documentação, e outras senhas antigas no histórico do git.

Como resolver:

- **Revogar o token do Nx Cloud imediatamente** e passar a usá-lo só por variável de ambiente (segredo do GitHub no CI).
- Trocar todas as senhas que apareceram no repositório. Apagar o arquivo não basta: o que já foi publicado deve ser tratado como vazado.
- Nos arquivos de configuração, usar apenas variáveis de ambiente, sem senha padrão. Documentar as variáveis num arquivo de exemplo com valores fictícios.
- Fazer a aplicação se recusar a iniciar se a senha do banco estiver vazia ou for uma senha padrão conhecida.
- Ativar a varredura de segredos do GitHub, que bloqueia novos envios com chaves.

7. Guardar senhas com hash

Problema (A-X1): as senhas dos usuários ficam salvas no banco exatamente como foram digitadas. Se o banco vazar, todas as senhas vazam juntas.

Como resolver:

- Salvar as senhas com hash (BCrypt), que embaralha a senha de forma irreversível. No login, o servidor compara o hash, não a senha.
- Para as contas existentes, converter a senha no próximo login bem-sucedido e pedir troca de senha para quem não entrar dentro de um prazo.

8. Ajustes menores

- **Imagens de banner (B-04):** hoje aceitam qualquer endereço externo, o que permite rastrear quem vê o card. Aceitar só endereços seguros (https) de sites permitidos ou fazer o upload da imagem para o próprio sistema. Não há XSS aqui, porque o Angular já protege esse campo.
- **Mensagens de erro (B-X1):** hoje os erros do banco chegam ao usuário com nomes de tabelas. Mostrar uma mensagem genérica com um código de referência e guardar o detalhe só no log do servidor.
- **Swagger (I-03):** a documentação interativa da API está ligada em produção. Desligá-la fora do ambiente de desenvolvimento.

Ordem sugerida

Imediato (poucos dias):

1. Revogar o token do Nx Cloud.
2. Remover o login sem senha.
3. Bloquear temporariamente as rotas de edição e exclusão, até a autenticação ficar pronta.
4. Trocar as senhas vazadas.

Curto prazo (próxima sprint):

1. Implementar a autenticação no servidor (item 1).
2. Pegar usuário e instituição do token em todas as rotas (itens 3 e 4).
3. Conferir o dono em todas as rotas por número (item 5).
4. Aplicar as permissões no servidor (item 4).
5. Passar a guardar as senhas com hash (item 7).

Médio prazo:

1. Os ajustes menores (item 8) e a limpeza de senhas padrão (item 6).
2. Como segunda barreira, avaliar o isolamento por instituição direto no banco de dados (Row Level Security do PostgreSQL).

A autenticação precisa vir primeiro no curto prazo: sem ela, o servidor não tem de onde tirar quem é o usuário para fazer as outras verificações.

Como saber que foi corrigido

- Um pedido à API sem token é recusado.
- Um usuário da instituição A não consegue ver, alterar ou apagar nada da instituição B.
- Um usuário sem permissão de administrador é bloqueado nas ações administrativas, mesmo chamando a API diretamente.
- O login sem senha não funciona mais.
- Não há nenhuma senha ou token nos arquivos do repositório, e os antigos foram trocados.
- As senhas no banco aparecem como hash, não como texto.
- Cada um desses pontos tem um teste automático que roda no CI.

O que já está bom e deve ser mantido

A auditoria também confirmou pontos fortes: não há risco de SQL injection (as consultas usam parâmetros), não há XSS no frontend (o Angular trata todo texto exibido), o ambiente de produção não guarda senhas no código e o CI usa permissões mínimas. As correções acima não devem mexer nesses pontos.